Department of Computer Science and
Artificial Intelligence,
National Pingtung University

Final Report of Capstone Project

Design and Implementation of an Information and Communication Security Mechanism Integrating Deep Learning and Image Encryption Techniques

Team Members: Chin-An Liu (CBE109013)
Yu-Ming Huang (CBE109038)
Huang-Hua Huang (CBE109035)

Supervisor: Dr. Cheng-Hsing Yang

December 2023

Abstract

Given the convenience of online shopping and the impact of the pandemic, the frequency of online shopping has significantly increased, while also exacerbating the threat of phishing websites targeting users' personal information. To address these issues, this project will integrate face recogntion, visual speech recognition, image encryption, and data embedding technologies to develop a secure verification website for third-party use, replacing the traditional mechanism of using One-Time Password(OTP) for online transactions. Through the development of this project, it is anticipated that the system will effectively block phishing websites from stealing users' personal information and subsequently using it to fraudulently charge their credit cards.

Table of Contents

Abstract i

Table of Contents ii

Table of Figures iii

List of Tables v

1. Introduction 1

2. Literature and theory 2

2-1 Face recogntion 2
2-1-3 Deep Convolutional Networks Selection 5
2-1-4 GoogleNet Inception Model 6
2-2 Visual Speech Recognition 8
2-2-1 Baseline VSR Model 9
2-2-2 Front-end Encoder 11
2-2-3 Back-end Encoder 11
2-2-4 Hybrid CTC and Transformer Decoder 11
2-3 Reversible Data Hiding 12
2-3-1 Sender 12
2-3-2 Receiver 15
3. Resaerch Methodology 16

3-1 Face Recognition 17
3-2 Visual Speech Recognition (VSR) 18
3-3 Reversible Data Hiding (RDH) 19
3-4 Implementation Flowchart 19
4. Results 22

4-1 Face Recognition 22
4-2 Visual Speech Recognition (VSR) 23
4-3 Reversible Data Hiding(RDH) 25
4-4 System Implementation Interface 25
5. Conclusion and Recommendations 28

References 29

Table of Figures

Figure 1-1: Conceptual diagram of credit card fraud on an e-commerce website 1

Figure 2-1: FaceNet Architecture 3

Figure 2-1-1: Triplet Loss Architecture 4

Figure 2-1-2: Inception module 7

Figure 2-1-3: Without adding 1×1convolution layer 7

Figure 2-1-4: Adding 1×1convolution layer 8

Figure 2-1-5: Auxiliary Classifiers 8

Figure 2-2-1: Visual Speech Recognition model flowchart 9

Figure 2-2-2: (a) Visual Speech Recognition model, (b) Visual Speech Recognition encoder architecture 10

Figure 2-2-3: Front-end Encoder architecture 11

Figure 2-3-1: Data Hiding and Extraction flowchart 12

Figure 2-3-2: (a) 3 × 3 Original image, (b) 5 × 5 Augumented image 12

Figure 2-3-3: Four 3 × 3 augumented image formed from Figure 2-3-2(b) 13

Figure 2-3-4: Four 3 × 3 blocks 13

Figure 2-3-5: Image after embedding information into the original image 14

Figure 2-3-6: Sequential extraction of secret bits and update of seed pixel values 15

Figure 2-3-7: Restored image 16

Figure 3-1-1: Standard Face Recognition Process 17

Figure 3-3-1: Reversible Data Hiding flowchart 19

Figure 3-4-1: Four-Party Transaction Diagram 20

Figure 4-1-1: Loss function variation curve 23

Figure 4-1-2: Dataset Evaluation ROC Curve 23

Figure 4-3-1: Original and Decrypted images 25

Figure 4-4-1: Bank account Registration Interface 26

Figure 4-4-2: Shopping website account Registration Interface 26

Figure 4-4-3: User ordering Interface 27

Figure 4-4-4: Bank and Shopping website Transaction Information Interface 27

List of Tables

Table 1. Results under Different Conditions on the LRS3 Dataset 24

Table 2. Results of Different Methods on the LRS3 Dataset 24

Table 3. Embedding Capacity and Embedding Rate of Restored Images 25

1. Introduction

Due to the sophistication of phishing website technology, users often unknowingly disclose their personal information and credit card details, leading to credit card fraud by hackers. The process is illustrated in Figure 1-1. For banks, the occurrence of credit card fraud not only results in financial losses but also involves a complex and time-consuming post-incident handling process. Therefore, how to quickly and accurately determine whether a transaction was initiated by the user has become an important research topic.

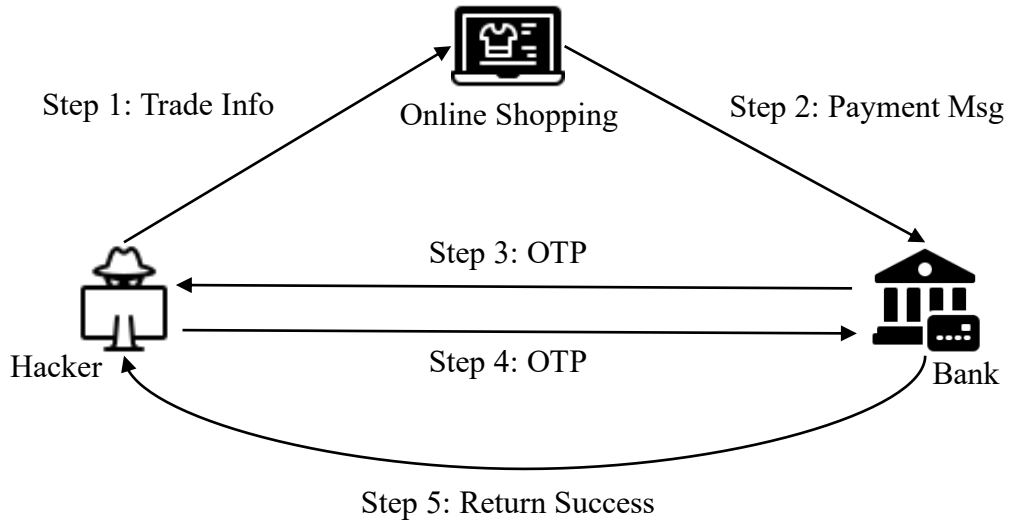


Figure 1-1: Conceptual diagram of credit card fraud on an e-commerce website

Figure 1-1 illustrates the process of an illegal credit card fraud by a hacker, involving the roles of the hacker, the shopping website, and the bank. The relationships between these roles are as follows:

Step 1: Hackers illegally obtain user information through phishing websites and then use it to make unauthorised purchases on shopping websites.

Step 2: After receiving the order information, the shopping website sends the transaction details to the bank.

Step 3: The bank sends a verification code (OTP) via SMS to the user's mobile phone number.

Step 4: The hacker intercepts the verification code and successfully enters it to complete the identity verification.

Step 5: The bank confirms the transaction as successful, and the hacker completes the credit card fraud.

By combining face recogntion and visual voice recognition technology, the above issues can be resolved. In the face recogntion component, the user's facial image replaces the traditional credit card security code to verify whether the transaction was performed by the user. In the visual speech recognition part, the user's lip-reading verification video replaces the traditional verification code to verify whether the transaction was performed by a real person, rather than a 2D facial image. The facial feature values extracted in face recogntion will be embedded along with other confidential information into a specific image to achieve encryption, and will subsequently serve as relevant information for the bank to verify transactions.

2. Literature and theory

2-1 Face recogntion

This study adopts the method proposed by Schroff, et al. [1] to extract feature vectors from a pair of input face images using a Convolutional Neural Network (CNN). The smaller the Euclidean distance between the feature vectors of two facial images, the higher the probability that the two individuals are the same person. Once the face feature extraction model is trained, face recogntion becomes a problem of calculating the Euclidean distance between two face feature vectors and comparing it with a threshold value.

In this paper, Facenet uses ZFNet(Zeiler and Fergus [2]) and GoogleNet V1(Szegedy, et al. [3]) as neural networks, and then uses Triplet Loss to train the model to output 128-

dimensional eigenvalues for facial recogntion purposes. In other words, the input face image is processed by the neural network $f(x)$, converting the image x into a vector in the $\mathbb{R}^d$ space。The following section introduces the entire FaceNet architecture, as shown in Figure 2-1:

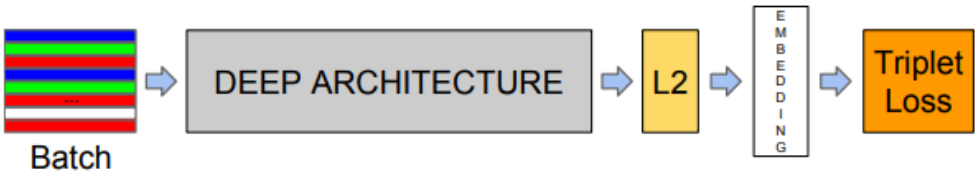


Figure 2-1: FaceNet Architecture

According to Figure 2-1，the system first inputs a face image, extracts features through an intermediate model, then applies L^2 regularisation for feature enhancement, and then the results are passed into the embedding layer. Finally, the triplet loss is used to calculate the differences for model training.

2-1-1 Triplet Loss

Triplet loss is a loss function in deep learning, primarily used for samples with small differences, such as faces. A triplet consists of two face images from the same person and a face image from another person. During model training, the Euclidean distance between face images from the same person should be smaller than the Euclidean distance between face images from different people. In Sun, et al. [4], all facial projections of a person are mapped to a single point in the embedding space, and the features of each person are kept at a certain distance from others. Embedding refers to mapping an image x into a d-dimensional Euclidean space. In this paper, the embedding is restricted to a d-dimensional hypersphere, i.e. $\|f(x)\|_2 = 1$. Anchor, Positive, and Negative samples are the anchor face image x_i^a, positive face image x_i^p, and negative face image x_i^n, respectively. It is necessary to ensure that the distance between the anchor and positive sample images is less than the distance between the anchor and negative sample image. Therefore, this paper aims to achieve

equations (1), (2).

$$\left\|f(x_i^a) - f(x_i^p)\right\|_2^2 + \alpha < \|f(x_i^a) - f(x_i^n)\|_2^2 \quad (1)$$

$$\forall \left(\mathrm{f}(x_i^a), \mathrm{f}(x_i^p), \mathrm{f}(x_i^n)\right) \in T \quad (2)$$

α: A margin that forces the model to learn, making the distance between anchor points and negative samples larger.

T: The training set of all triplets, with cardinality N

Triplet loss primarily minimises the distance between the anchor and positive samples with the same class, while maximising the distance between the anchor and negative samples with different classes. The following is a diagram of the triplet loss architecture, as shown in Figure 2-1-1:

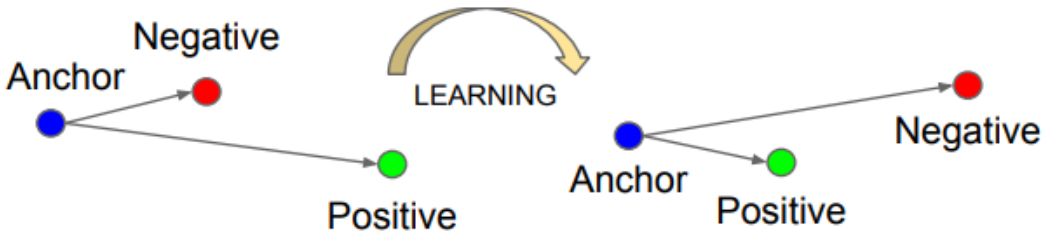


Figure 2-1-1: Triplet Loss Architecture

The minimized loss is $L = \sum_i^N \left[\left\|f(x_i^a) - f(x_i^p)\right\|_2^2 - \|f(x_i^a) - f(x_i^n)\|_2^2 + \alpha\right]_+$, which will generate all matching triples. However, these triplets may easily satisfy the above equation, and these triplets will not be helpful for training because they will still pass through the model, leading to a sudden decrease in convergence speed. Therefore, to enable the model to converge quickly, the selection of triplets is particularly important.

2-1-2 Triplet Selection

To prevent the features of samples from clustering into a very small space, the distance between the anchor and negative samples from different classes should be

relatively farther than the distance between the anchor and positive samples of the same class. This means that for a given x_i^a, the x_i^p(hard positive: positive samples of the same class are far from the anchor point) and x_i^n(hard negative: negative samples from different classes that are very close to the anchor point) are selected. Formally, the hard positive is chosen as: $argmax_{x_i^a}\left\|f(x_i^a) - f(x_i^p)\right\|_2^2$ and the hard negative is chosen as: $argmin_{x_i^n}\|f(x_i^a) - f(x_i^n)\|_2^2$. However, calculating the argument that reaches the maximum value(argmax) value or minimum value(argmin) over the entire dataset is not feasible. Therefore, there are two methods to avoid this issue.

- Generate offline triplets: After every n training iterations, calculate the positive samples that are far from the anchor point and the negative samples that are close to the anchor point in a subset.
- Generate online triplets: Take a large number of mini-batches, and within each mini-batch, calculate the positive samples that are far from the anchor point and the negative samples that are close to the anchor point.

In this paper, large mini-batches are used with online generation, and only the following arguments are considered: the minimum value (argmin) and the maximum value (argmax) within each mini-batch. To obtain meaningful distances between anchor points and positive samples, ensuring that each person has the minimum number of samples. In each mini-batch, select negative samples that are close to the anchor points based on the distances between all anchor points and positive samples. In this way, this approach can converges faster than selecting positive samples that are far from the anchor points.

2-1-3 Deep Convolutional Networks Selection

In this paper, the ZF-Net Model and GoogleNet Inception Model were used, with

an additional 1×1×d convolution layer added at the end to enable the model to output a (1, 1, d) vector. According to the experimental results, using the Inception Model with an input value of (224, 224, 3) and a 128-dimensional output vector yields the best accuracy.

2-1-4 GoogleNet Inception Model

GoogleNet Inception Model consists of multiple small basic modules connected in series to form a large network. These small network modules are Inception module.

- Inception module

  Initially, inception module adopted the structure shown in Figure 2-1-2(a), with detailed content as illustrated in Figure 2-1-2(c). In the GoogleNet paper, three different sizes of convolutional and max pooling layers are used to extract different features, which are then concatenated along the channel axis. This approach of increasing the network width enables the capture of more image features and details. Later, to reduce the number of training parameters, a 1×1 convolutional layer was added to the original architecture. The improved Inception Module first reduces the number of output channels using a 1×1 convolution layer before connecting to the original convolution layer. In addition, since pooling layers cannot reduce the number of channels, the input feature map is passed through a 3×3 convolution layer followed by a 1×1 convolution layer to reduce the number of output channels, as shown in Figure 2-1-2(b):

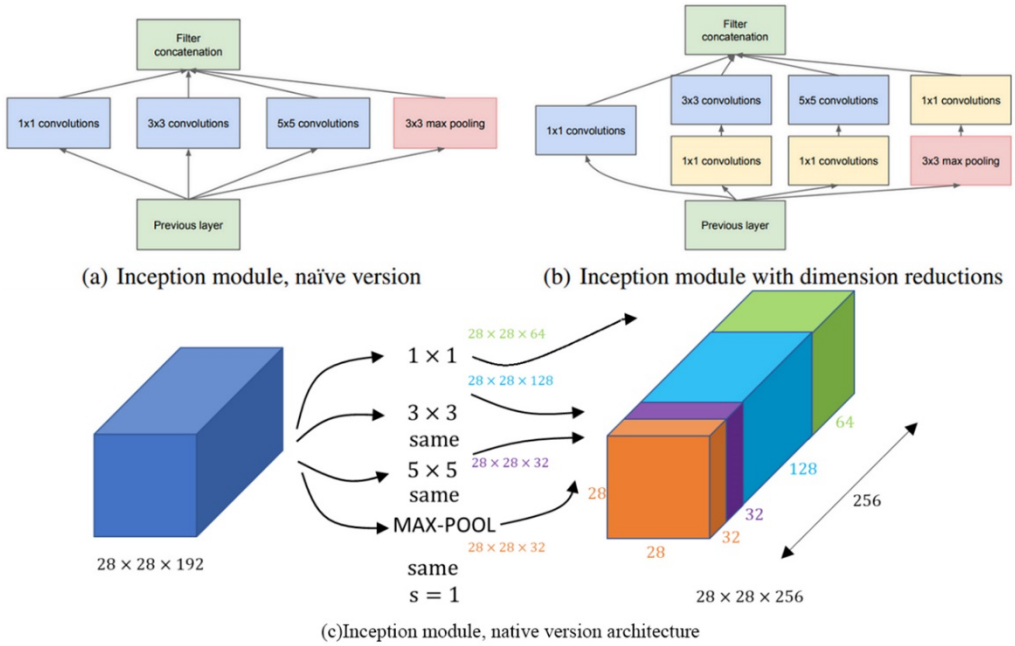


Figure 2-1-2: Inception module

- Reducing the number of training parameters

The original architecture is shown in Figure 2-1-3, which includes a single 5×5 convolution layer with a total of 28×28×16×5×5×192 = 120 (approximately 120 million) parameters. After adding a 1×1 convolutional layer as proposed in the GoogleNet paper, the number of parameters was reduced to 28×28×16×192 + 28×28×32×5×5×16 = 2.4 + 10 = 12.4 (approximately 12.4 million), as shown in Figure 2-1-4. Therefore, the improved Inception module can effectively reduces the number of training parameters.

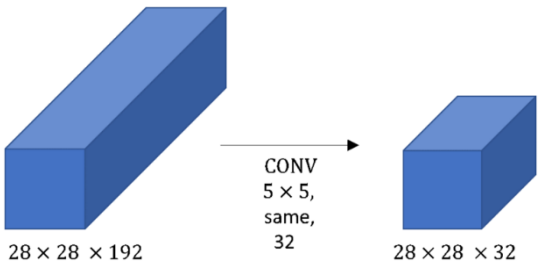


Figure 2-1-3: Without adding 1×1convolution layer

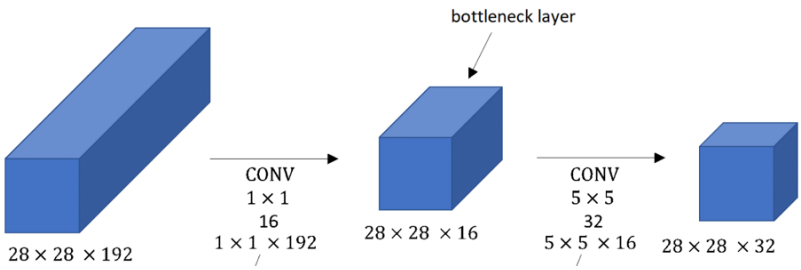


Figure 2-1-4: Adding 1×1convolution layer

- Auxiliary Classifiers

Since GoogleNet is a deep neural network, it may encounter the vanishing gradient problem. Therefore, to address these issues, GoogleNet incorporates two auxiliary classifiers. When calculating the total loss, the loss from the classifier is multiplied by 0.3 and added to the total loss. In addition, during actual testing, the auxiliary classifiers are removed from the network. The following figures show the two auxiliary classifiers, as shown in Figure 2-1-5:

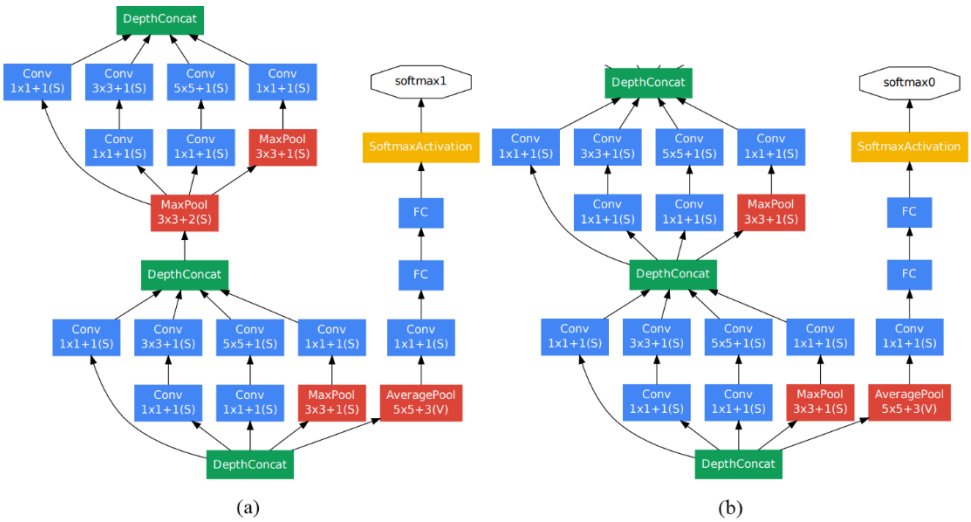


Figure 2-1-5: Auxiliary Classifiers

2-2 Visual Speech Recognition

Visual Speech Recognition(VSR, Ma, et al. [5]) is a speech recognition technique that inputs lip-reading videos into a natural language model and outputs predicted word

sequences. First, we introduce the basic visual speech recognition model. To improve the model's accuracy, Ma, et al. [5] adds an encoder and decoder to the original model. The improved model architecture is mainly divided into four parts: the Front-end Encoder, Visual Speech Recognition Encoder (VSR Encoder), Back-end Encoder, and Hybrid Decoder (Hybrid CTC and Transformer Decoder), as shown in Figure 2-2-1:

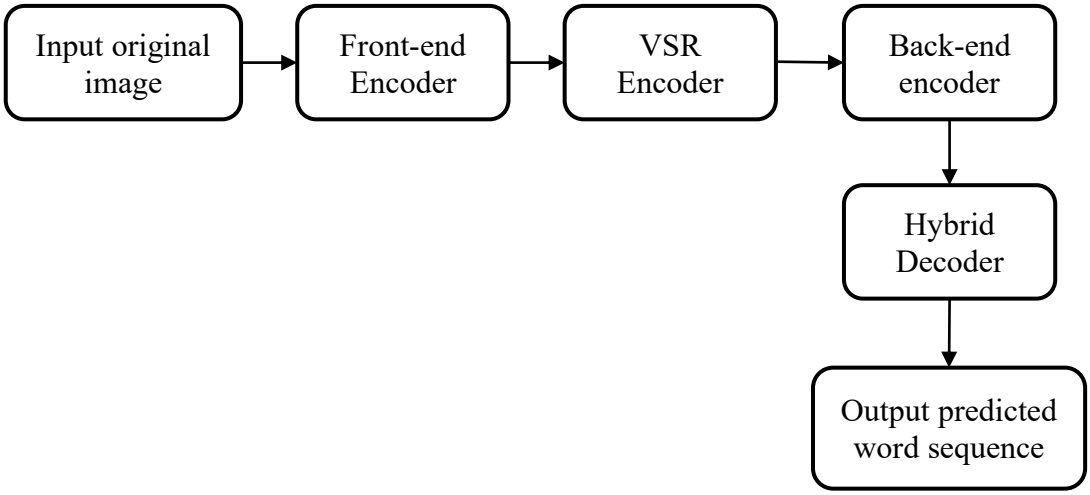
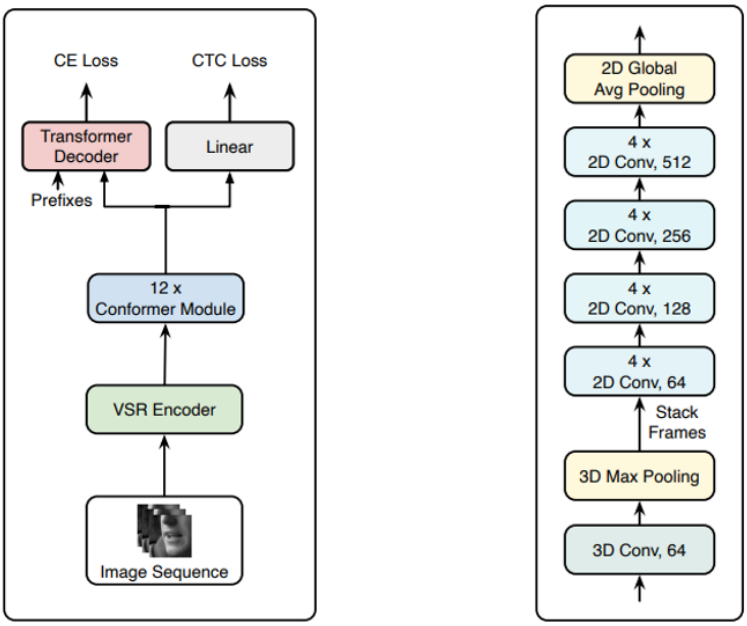


Figure 2-2-1: Visual Speech Recognition model flowchart

2-2-1 Baseline VSR Model

Five input images are processed by a 2D convolution layer, and the resulting features are passed into a 2D ResNet-18 (VSR Encoder), as shown in Figure 2-2-2(b). Then, the output is passed to a 12-layer conformer model, followed by a Transformer model for decoding, with the overall architecture illustrated in Figure 2-2-2(a). The model is trained using a combination of end-to-end Connectionist Temporal Classification loss (CTC loss) and self-attention mechanisms. During training, data augmentation is performed through random cropping and image flipping. In addition, time masking is also used for data augmentation. Although it is commonly used in Automatic Speech Recognition (ASR) models, it is also applied in visual speech recognition to encourge the model to more effectively use of contextual information,

thereby improving lip reading accuracy.



VSR model VSR encoder

Figure 2-2-2: (a) Visual Speech Recognition model, (b) Visual Speech Recognition encoder architecture

- Connectionist Temporal Classification loss (CTC loss)

  Commonly used for calculating losses between continuous time series and target sequences.

- Data augumentation

  Used to increase the amount of training data and reduce the occurrence of overfitting.

- Time masking

  By selecting a few frames from n consecutive frames and replacing the intermediate frames, this allows the model to utilise contextual information more effectively, thereby improving the accuracy of lip movement detection.

2-2-2 Front-end Encoder

The input size is B×T×H×W, where B, T, H, and W represent the batch size, number of frames, image height, and image width, respectively. After being processed by a 3D convolution layer, the input is passed into ResNet-18. The visual features at the top of the residual blocks are aggregated along the spatial dimension by a global average pooling layer, the architecture is shown in Figure 2-2-3:

Component Name	Layer Type	Input Size	Output Size
Stem_1	Conv 3D, 5×7^2, 64	[B, 1, T_v, 88, 88]	[B, 64, T_v, 44, 44]
	3D Max Pooling, 1×3^2	[B, 64, T_v, 44, 44]	[B, 64, T_v, 22, 22]
Reshape	-	[B, 64, T_v, 22, 22]	[B×T_v, 64, 22, 22]
Residual Block_2	$\begin{bmatrix} \text{Conv 2D}, 3^2, 64 \\ \text{Conv 2D}, 3^2, 64 \end{bmatrix} \times 2$	[B×T_v, 64, 22, 22]	[B×T_v, 64, 22, 22]
Residual Block_3	$\begin{bmatrix} \text{Conv 2D}, 3^2, 128 \\ \text{Conv 2D}, 3^2, 128 \end{bmatrix} \times 2$	[B×T_v, 64, 22, 22]	[B×T_v, 128, 11, 11]
Residual Block_4	$\begin{bmatrix} \text{Conv 2D}, 3^2, 256 \\ \text{Conv 2D}, 3^2, 256 \end{bmatrix} \times 2$	[B×T_v, 128, 11, 11]	[B×T_v, 256, 6, 6]
Residual Block_5	$\begin{bmatrix} \text{Conv 2D}, 3^2, 512 \\ \text{Conv 2D}, 3^2, 512 \end{bmatrix} \times 2$	[B×T_v, 256, 6, 6]	[B×T_v, 512, 3, 3]
Aggregation	2D Global Average Pooling	[B×T_v, 512, 3, 3]	[B×T_v, 512, 1, 1]
Reshape	-	[B×T_v, 512, 1, 1]	[B, 512, T_v]

Figure 2-2-3: Front-end Encoder architecture

2-2-3 Back-end Encoder

The process begins with a positional embedding module, followed by 12 layer conformer blocks. The positional embedding module is a linear layer that maps the features from ResNet-18 to a 256-dimensional space; The conformer block consists of a feed-forward module, a self-attention module, a convolution module, and a second feed-forward module.

2-2-4 Hybrid CTC and Transformer Decoder

It consists of an embedding module and a set of residual multi-head attention blocks. Given the encoded sequence and the prefix of the target sequence, the indexes from 1 to $l-1$ (where l is the index of the target length) of the prefixes are projected into embedding vectors. These embedding vectors are then fed into a stack

of multi-head attention blocks, each containing a self-attention module, an encoder–decoder module, and a feed-forward module.

2-3 Reversible Data Hiding

Reversible data hiding is a technique that hides secret information in images and successfully extracts the secret information while fully restoring the images. It can be applied in fields such as finance, medicine, and military. This section introduces the working mechanism proposed by Singh and Singh [6], and discusses it in two parts: Sender and Reciever, as shown inFigure 2-3-1:

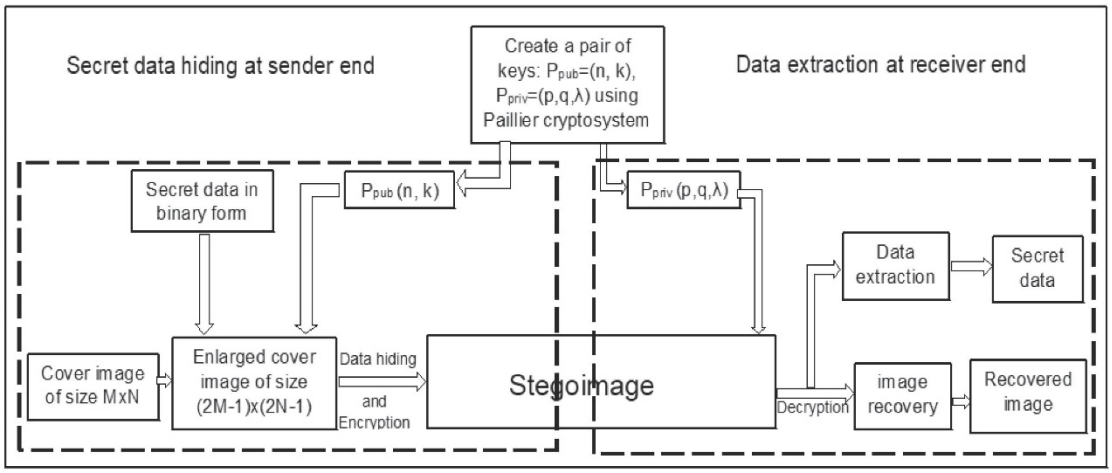


Figure 2-3-1: Data Hiding and Extraction flowchart

2-3-1 Sender

Step 1: Scale up the input M×N-sized original image to a scaled-up image of size 2M-1×2N-1, as shown in Figure 2-3-2, and divide the pixel values into seed pixels P_i and non-seed pixels, Q_i.

P_1=110	P_2=159	P_3=162
P_4=240	P_5=242	P_6=213
P_7=230	P_8=251	P_9=224

(a)

P_1=110	Q_1=1	P_2=159	Q_2=1	P_3=162
Q_3=1	Q_4=1	Q_5=1	Q_6=1	Q_7=1
P_4=240	Q_8=1	P_5=242	Q_9=1	P_6=213
Q_{10}=1	Q_{11}=1	Q_{12}=1	Q_{13}=1	Q_{14}=1
P_7=230	Q_{15}=1	P_8=251	Q_{16}=1	P_9=224

(b)

Figure 2-3-2: (a) 3 × 3 Original image, (b) 5 × 5 Augumented image

Step 2: Divide the augumented image into overlapping blocks of size 3×3, as shown in Figure 2-3-3, and perform data embedding on each block.

P_1=110	Q_1=1	P_2=159
Q_3=1	Q_4=1	Q_5=1
P_4=240	Q_8=1	P_5=242

(a)

P_2=159	Q_2=1	P_3=162
Q_5=1	Q_6=1	Q_7=1
P_5=242	Q_9=1	P_6=213

(b)

P_4=240	Q_8=1	P_5=242
Q_{10}=1	Q_{11}=1	Q_{12}=1
P_7=230	Q_{15}=1	P_8=251

(c)

P_5=242	Q_9=1	P_6=213
Q_{12}=1	Q_{13}=1	Q_{14}=1
P_8=251	Q_{16}=1	P_9=224

(d)

Figure 2-3-3: Four 3 × 3 augumented image formed from Figure 2-3-2(b)

Step 3: Replace the central pixel value of each block with the average value of all seed pixels within the block, as shown in Figure 2-3-4.

P_1=110	Q_1=1	P_2=159
Q_3=1	Q_4=187	Q_5=1
P_4=240	Q_8=1	P_5=242

(a)

P_2=159	Q_2=1	P_3=162
Q_5=1	Q_6=194	Q_7=1
P_5=242	Q_9=1	P_6=213

(b)

P_4=240	Q_8=1	P_5=242
Q_{10}=1	Q_{11}=240	Q_{12}=1
P_7=230	Q_{15}=1	P_8=251

(c)

P_5=242	Q_9=1	P_6=213
Q_{12}=1	Q_{13}=232	Q_{14}=1
P_8=251	Q_{16}=1	P_9=224

(d)

Figure 2-3-4: Four 3 × 3 blocks

Step 4: Convert the secret message into binary format and extract the binary data from left to right in sequence to perform the hiding operation.

Step 5: For each block, perform an even parity check on the central pixel value and the 1-bit secret message from top-left to bottom-right. If the two values are

different, increment the central pixel value of the block by 1.

Step 6: For all seed pixels within each block, embed 2-bit-sizedsecret message, i.e., embed the 2-bit-sized secret message V1 into the pixel value PL1 of block BL1, using the following equations (3), (4), where PL1 refers to pixel value and PE refers Pailler encryption. If equation (3) is executed, the adjacent non-seed pixel is multiplied by 2, indicating that the next seed pixel to be embedded is located below or to the left of this seed pixel. If equation (4) is executed, the adjacent non-seed pixel is multiplied by 3, indicating that the next seed pixel to be embedded is located to the right or above this seed pixel.

If PL1 % 2 != 0

$$stegoPL1 = \left(PE\left((PL1-1)\times 2\right)PE(V1)PE(512)\right) mod\ N^2 \tag{3}$$

else

$$stegoPL1 = \left(PE(PL1\times 2)PE(V1)\right) mod\ N^2 \tag{4}$$

Step 7: Perform Step 6 on each block in a clockwise direction. The final result is shown in Figure 2-3-5.

P_1=PE(220)	Q_1=PE(1)	P_2=PE(2174)	Q_2=PE(3)	P_3=PE(327)
Q_3=PE(1)	Q_4=PE(187)	Q_5=PE(2)	Q_6=PE(194)	Q_7=PE(1)
P_4=PE(1473)	Q_8=PE(3)	P_5=PE(4387)	Q_9=PE(6)	P_6=PE(1874)
Q_{10}=PE(1)	Q_{11}=PE(241)	Q_{12}=PE(1)	Q_{13}=PE(232)	Q_{14}=PE(1)
P_7=PE(463)	Q_{15}=PE(2)	P_8=PE(2029)	Q_{16}=PE(1)	P_9=PE(448)

Figure 2-3-5: Image after embedding information into the original image

Note: In the referenced paper, homomorphic encryption (Paillier cryptosystem) is applied to the processed image data. However, since performing

homomorphic encryption significantly increases execution time, this encryption mechanism is not incorporated in the implementation of this project.

2-3-2 Receiver

Step 1: Divide the received image into overlapping blocks of size 3×3, and perform data extraction and image recovery on each block.

Step 2: Convert the centre pixel value of each block into binary, then extract the rightmost bit value from the rightmost position, following the order shown in Figure 2-3-6 (a), (b), (c), and (d) to obtain the leftmost 4 bits of the confidential data.

P_1= 220	Q_1= 1	P_2=2174	Q_2=3	P_3=327
Q_3=1	Q_4=187	Q_5=2	Q_6=194	Q_7=1
P_4=1473	Q_8=3	**P_5=1937**	Q_9=2	**P_6=936**
Q_{10}=1	Q_{11}=241	Q_{12}=1	Q_{13}=232	Q_{14}=1
P_7=463	Q_{15}=2	**P_8=1014**	Q_{16}=1	**P_9=224**

(a)

P_1= 220	Q_1= 1	P_2=2174	Q_2=3	P_3=327
Q_3=1	Q_4=187	Q_5=2	Q_6=194	Q_7=1
P_4=481	Q_8=3	**P_5=968**	Q_9=2	P_6=936
Q_{10}=1	Q_{11}=241	Q_{12}=1	Q_{13}=232	Q_{14}=1
P_7=230	Q_{15}=2	**P_8=251**	Q_{16}=1	P_9=224

(b)

P_1= 220	Q_1= 1	**P_2=831**	Q_2=3	**P_3=162**
Q_3=1	Q_4=187	Q_5=2	Q_6=194	Q_7=1
P_4=481	Q_8=3	**P_5=484**	Q_9=2	**P_6=213**
Q_{10}=1	Q_{11}=241	Q_{12}=1	Q_{13}=232	Q_{14}=1
P_7=230	Q_{15}=2	P_8=251	Q_{16}=1	P_9=224

(c)

P_1=110	Q_1= 1	**P_2=159**	Q_2=3	P_3=162
Q_3=1	Q_4=187	Q_5=2	Q_6=194	Q_7=1
P_4=240	Q_8=3	**P_5=242**	Q_9=2	P_6=213
Q_{10}=1	Q_{11}=241	Q_{12}=1	Q_{13}=232	Q_{14}=1
P_7=230	Q_{15}=2	P_8=251	Q_{16}=1	P_9=224

(d)

Figure 2-3-6: Sequential extraction of secret bits and update of seed pixel values

Step 3: For each block, extract the 2-bit-sized secret message hidden by the transmitter from all seed pixels in a clockwise direction. The calculation equations are (5), (6). If the non-seed pixel adjacent to the seed pixel is divisible by 2 or 3, execute equation (5), otherwise execute equation (6).

If PL2 was obtained using (3),(4)

$$NP = (PL2 - V2 - 512) / 2 + 1 \quad (5)$$

else

$$NP = (PL2 - V2) / 2 \quad (6)$$

V2=PL2 mod 4

Step 4: Perform Step 3 sequentially from the top-left to the bottom-right for each block. The final result is shown in Figure 2-3-7.

P_1=110	P_2=159	P_3=162
P_4=240	P_5=242	P_6=213
P_7=230	P_8=251	P_9=224

Figure 2-3-7: Restored image

3. Resaerch Methodology

This project is divided into three main parts: Face recogntion, Visual Speech Recognition, and Reversible Data Hiding. First, face recogntion is used to determine whether the transaction was performed by the cardholder. Second, visual speech recognition is used to ensure that the user is a real person. Finally, reversible information hiding is employed to prevent data from being intercepted during transmission. The following sections will introduce each part in detail.

3-1 Face Recognition

This section uses the FaceNet model, with NN2 selected as the backbone network, and CASIA-WebFace (Yi, et al. [7]) as the training data for facial feature vector learning. The trained FaceNet model weights are used for face recognition. During user registration, the model generates three sets of 128-dimensional vectors representing the features of the user's frontal face, which are stored in the database. For subsequent login attempts, the user's image is input into the FaceNet model to produce a 128-dimensional feature vector, which is compared with the three registered vectors by omputing the Euclidean distance. If the Euclidean distance is below the predefined threshold, the recognition is considered successful. The process is illustrated Figure 3-1-1:

1. Input face image with size of 224×224
2. Use MTCNN(Zhang, et al. [8]) for face detection and alignment
3. Extract face feature vectors by using the model
4. Perform L^2 standardisation and output a 128-dimensional face feature vector.

We divide the model output into two scenarios. During training, we combined the Cross-Entropy Loss function and Triplet Loss function as the overall loss functions. The cross-entropy loss function is primarily used for face classification and assists the triplet loss function in convergence. To use the cross-entropy loss function, a classifier must be constructed, applying a full-connected layer to the output for classification. During inference, the classifier is not required; instead, the model directly outputs a 128-dimensional feature vector.

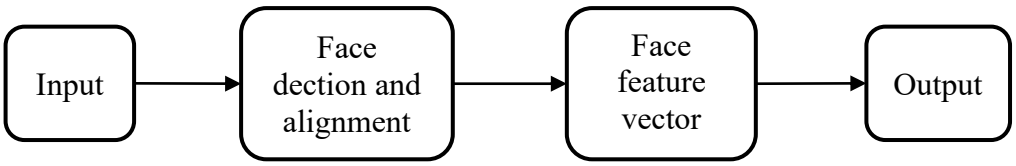


Figure 3-1-1: Standard Face Recognition Process

We have found that while using face recognition technology to replace traditional username and password login functionality can effectively enhance convenience, relying solely on face recognition technology may pose risks from deepfake technology. Therefore, this project incorporates visual speech recognition to address this issue.

3-2 Visual Speech Recognition (VSR)

Given the maturity of generative speech technology, relying solely on voice recognition as a verification mechanism may not effectively block AI attacks. Therefore, this implementation will adopt visual speech recognition. When users perform shopping verification, they will receive a verification code (e.g., Vocabulary) sent by the system. Users must face the webcam and repeat the verification code three times to complete the verification video recording. When the verification endpoint receives the verification video, it will first perform preprocessing on the video.

First, RetinaFace(Deng, et al. [9]) and Face Alignment Network(FAN, Bulat and Tzimiropoulos [10]) are used to detect 68 facial landmarks. These facial images are then subjected to similarity transformation to align them to a neutral reference frame, thereby removing translation and scaling differences.

Next, a 96×96 bounding box centred on the mouth centre is used to crop the region of interest (ROI) for the mouth. Finally, the cropped area is further converted into a grayscale image. The preprocessed video is input into a pre-trained visual speech recognition model to output the interpretation results. Finally, the interpretation results are compared with the verification code to confirm whether the operation was performed by a human. If the verification code comparison is successful, the information hiding stage is executed.

3-3 Reversible Data Hiding (RDH)

This part of the implementation will use reversible information hiding technology to hide transaction-related confidential information (including user card numbers, user feature values, transaction serial numbers, and transaction amounts) into images pre-agreed upon by the bank and the verification website for transmission. A text file is generated and then encrypted using RSA. The encrypted text file is then transmitted to the bank end, where it is decrypted using RSA. The secret data is extracted from the text file, and the image is restored. The restored image is compared with the original image to verify that no information was tampered with during transmission, as shown in Figure 3-3-1:

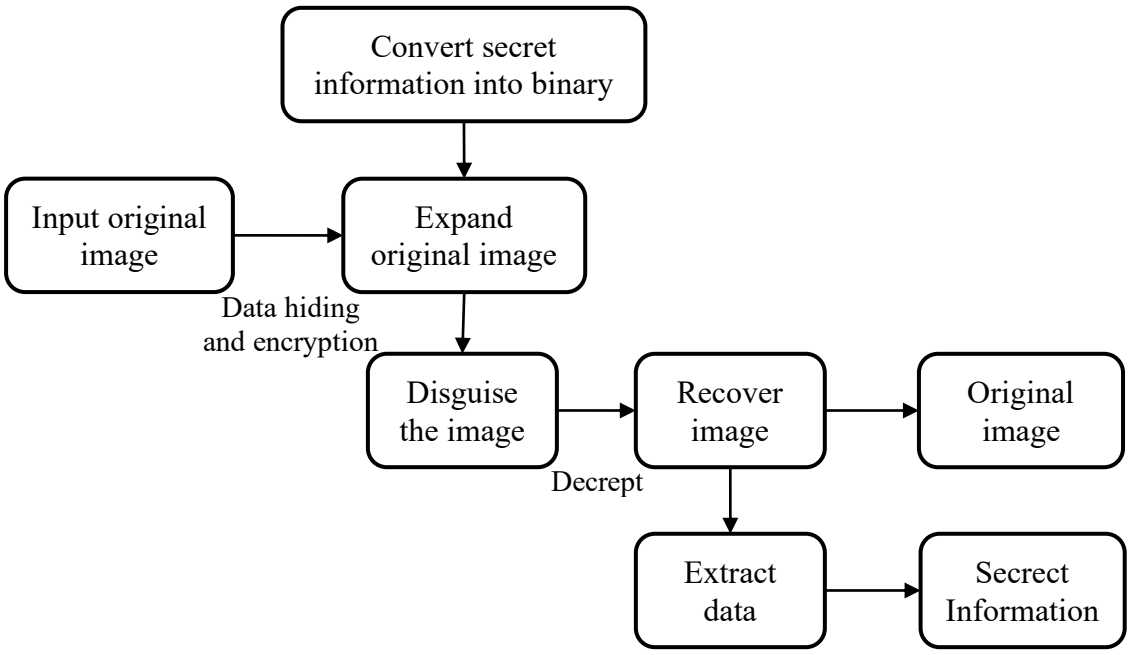


Figure 3-3-1: Reversible Data Hiding flowchart

3-4 Implementation Flowchart

This project aims to design and implement a third-party secure architecture website that acts as an intermediary between banks, users, and shopping websites by integrating face recognition, visual speech recognition, image encryption, and data embedding technologies. The overall process is illustrated in Figure 3-4-1:

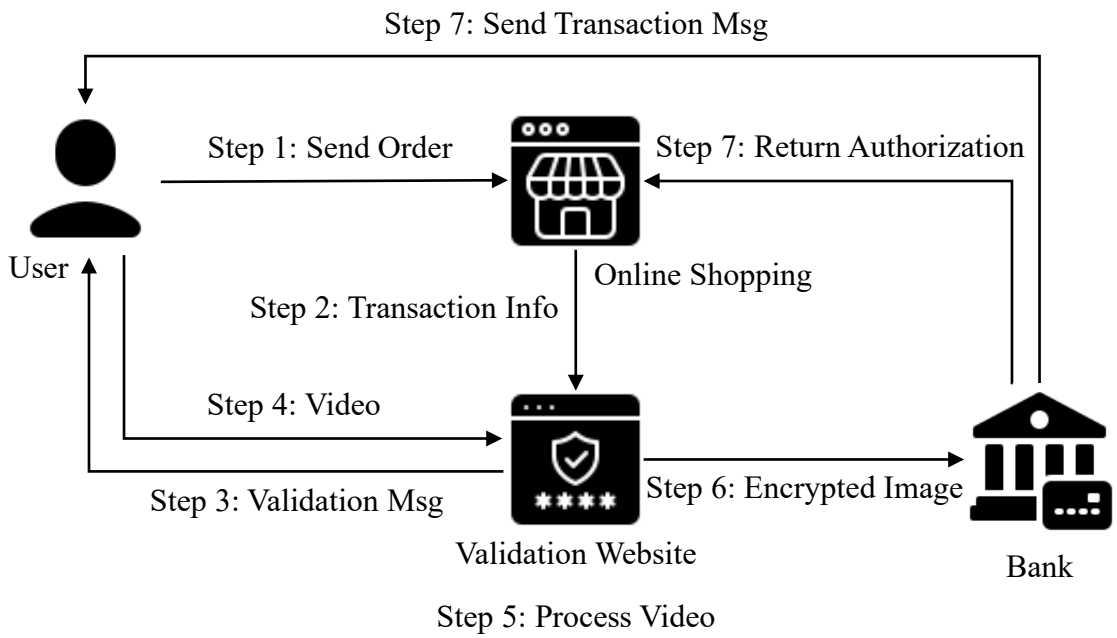


Figure 3-4-1: Four-Party Transaction Diagram

In Figure 3-4-1, there are four roles: the user end, the bank end, the verification website, and the shopping website. The relationships between these roles and the processing of information exchange are as follows:

Step 1: User submits an order to the e-commerce website

The user enters payment information, including the credit card number and t ransaction details.

Step 2: E-commerce website sends payment information to the verification website

The e-commerce website encrypts the payment information using asymmetric encryption (RSA algorithm) and transmits it to the verification website. The transmitted content includes the user's credit card number, transaction serial number, and transaction details.

Step 3: Verification website receives the payment information and generates a random verification code

1. The verification website receives the transaction information from the e-commerce website and decrypts it using the RSA algorithm.

2. The decrypted credit card number, transaction serial number, transaction details, and a randomly generated verification message are stored in the verification website's database.

3. The verification message generated in (2) is displayed on the verification website.

Step 4: User records a visual speech recognition video on the verification website

The user reads aloud the verification message displayed on the verification website to complete the visual speech recognition video.

Step 5: Verification website processes the video uploaded by the user

1. Using the credit card number and transaction serial number as search criteria, the system retrieves the stored verification message from the user's data.

2. The visual speech recognition model identifies the verification message spoken in the video.

3. If the identified message matches the stored verification message, the system proceeds to extract the facial image from the video.

4. A facial image is extracted from any frame of the verification video. The face recognition model extracts its feature vector, which, along with the payment information, is encrypted using asymmetric encryption (RSA algorithm) and sent to the bank. The transmitted content includes the credit card number, transaction serial number, transaction details, and facial feature vector.

Step 6: Bank decrypts the data and performs facial feature verification

1. During user registration (e.g., when applying for a credit card), the bank captures the user's frontal, left-profile, and right-profile facial images. The face recognition model extracts a feature vector from each image, which is stored in the database for Euclidean distance calculation.

2. The bank decrypts the facial feature vector and credit card number received from

the verification website.

3. Using the credit card information, the bank searches the database for the corresponding user data.

4. If the user data exists, the three stored facial feature vectors (frontal, left-profile, right-profile) are each compared with the facial feature vector received from the verification website by calculating their Euclidean distance. If the distance is below the predefined threshold, the transaction is approved; otherwise, it is rejected.

Step 7: Bank sends the transaction result to the e-commerce website and the user

1. If the transaction is approved, the bank sends an authorization success message to the e-commerce website, along with the transaction details (including payment amount, transaction time, transaction record, and e-commerce website name).

2. If the transaction is rejected, the bank sends an authorization failure message to the e-commerce website and notifies the user via SMS.

4. Results

4-1 Face Recognition

From the perspective of model convergence, according to Figure 4-1-1, the training and validation loss functions gradually decrease from approximately 9.5 and 9.3 to 2.2 and 1.5, respectively. This indicates that the model performs well during training. The decrease in training loss shows that the model fits the training data more effectively, while the decrease in validation loss indicates that the model can also correctly distinguish between the same person and different people on unseen data. Furthermore, there is no severe overfitting during the training process.

(Overfitting refers to a situation where the model performs very well on the training set but poorly on unseen data.)unseen data.

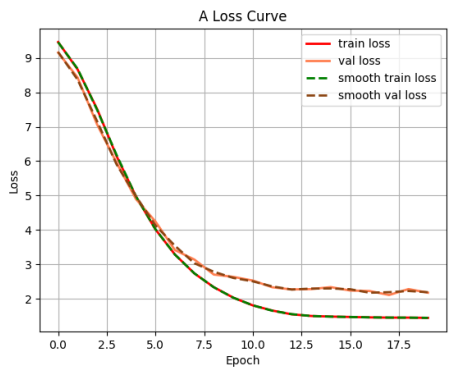
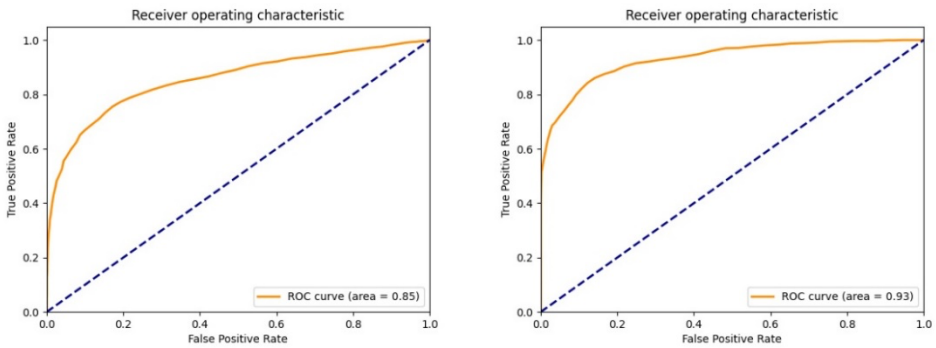


Figure 4-1-1: Loss function variation curve

From the perspective of model accuracy, we evaluate whether the model can accurately distinguish between the same person and different people. The trained model weights are tested on both the Labelled Faces in the Wild(LFW, Huang, et al. [11]) and a custom face dataset (including the faces of the three authors and other classmates). As shown in Figure 4-1-2(a), the accuracy reaches 79.162% on the LFW dataset, and in Figure 4-1-2(b), the accuracy reaches 92.3% on the custom dataset. Experimental results show that the model achieves high accuracy in face recognition.



(a) LFW Dataset (b) Other Dataset

Figure 4-1-2: Dataset Evaluation ROC Curve

4-2 Visual Speech Recognition (VSR)

The dataset used is a large-scale dataset for visual speech recognition (LRS3, Afouras, et al. [12]), consisting of:

- Pre-training set: 408 hours, 118,516 utterances

- Training and validation set: 30 hours, 31,982 utterances
- Test set: 0.9 hours, 1,321 utterances

The Word Error Rate (WER) is used to measure how close the predicted word sequence is to the target word sequence, as defined in Equation (7).

$$WER = \frac{S + D + I}{N} \tag{7}$$

where S, D, and I represent the number of substitutions, deletions, and insertions, respectively, and N is the total number of words in the target sequence.

Under different conditions, the model is trained using LRW + LRS3, and the WER is computed (Table 1).

Table 1. Results under Different Conditions on the LRS3 Dataset

Method	WER on LRS3
Our model	35.8±0.5
- Audio auxiliary task	36.6±0.3
- Visual auxiliary task	36.9±0.5
- Audio auxiliary task, visual auxiliary task	37.8±0.6
- Time masking	38.5±0.5
- Audio auxiliary task, visual auxiliary task, time masking	39.1±0.4

In addition, we compare the results of this VSR model with other visual verification models on the LRS3 dataset (Table 2).

Table 2. Results of Different Methods on the LRS3 Dataset

Method	Pre-training Set	Training Set	Training Sets Total Size (hours)	Mean±Std.	Best
Using Publicly Available Datasets					
KD+CTC [13]	VoxCeleb2^{clean}	LRS3	772	-	59.8
KD-seq2seq [14]	LRW+LRS2	LRS3	818	-	59.0
CM-seq2seq [10]	LRW	LRS3	595	-	43.3
Ours	-	LRS3	438	**38.6±0.4**	**37.9**
Ours	LRW	LRS3	595	**35.8±0.5**	**35.1**
Ours	LRW+LRS2	LRS3	818	**34.9±0.2**	**34.7**
Ours	LRW+LRS2+AVSpeech	LRS3	1 459	**32.1±0.3**	**31.5**

4-3 Reversible Data Hiding(RDH)

As shown in Figure 4-3-1, there is no visual loss between the original image and the recovered image after data embedding, confirming the reversibility of the data hiding process.

(a) Original image (b) Decrypted image

Figure 4-3-1: Original and Decrypted images

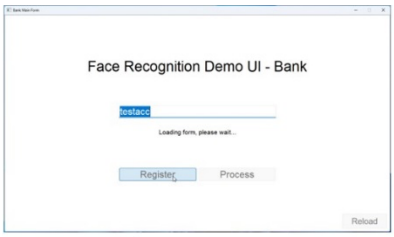
Table 3 presents the embedding capacity (EC) and embedding rate (ER) for each image used in this project. For all tested images (256×256), the embedding capacity is 585,209 bits, with an embedding rate of 2.24 bits per pixel.

Table 3. Embedding Capacity and Embedding Rate of Restored Images

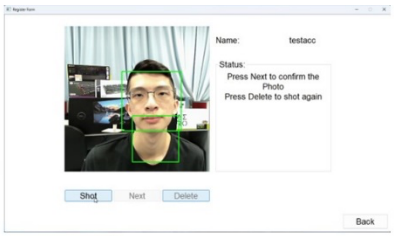
圖像(256×256)	藏量(EC)	嵌入率(ER)
Airplane	585209	2.24
Baboon	585209	2.24
Barbara	585209	2.24
Goldhill	585209	2.24
Pepper	585209	2.24
Sailboat	585209	2.24

4-4 System Implementation Interface

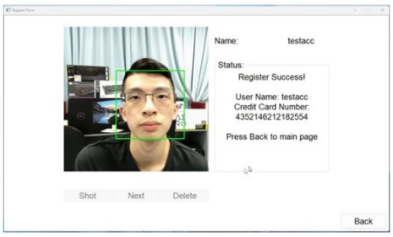
The system is implemented using PyQt5 integrated with MySQL to simulate the interfaces of the e-commerce website, verification website, and bank system. The process is divided into four parts: bank registration, e-commerce website registration, user ordering, and transaction processing.



(a) Input user name



(b) Take user's face images

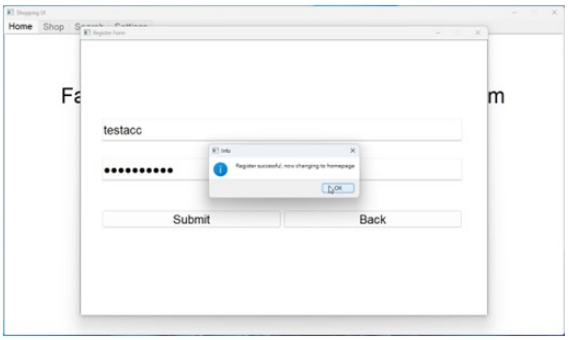


(c) User account registered successfully.

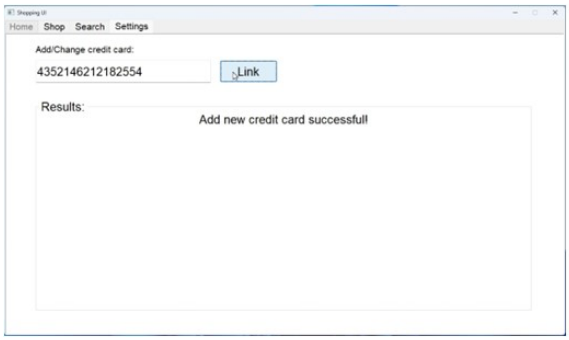
Figure 4-4-1: Bank account Registration Interface

Bank Registration:

As shown in Figure 4-4-1(a), the user enters their name at the bank terminal to begin account registration. Three frontal face images are then captured, aligned using MTCNN, and processed with the FaceNet model to extract facial feature vectors (Figure 4-4-1(b)). Finally, as shown in Figure 4-4-1(c), the three sets of facial feature vectors are stored in the bank's database, and the relevant account information is provided to the user.



(a) User account registered successfully.

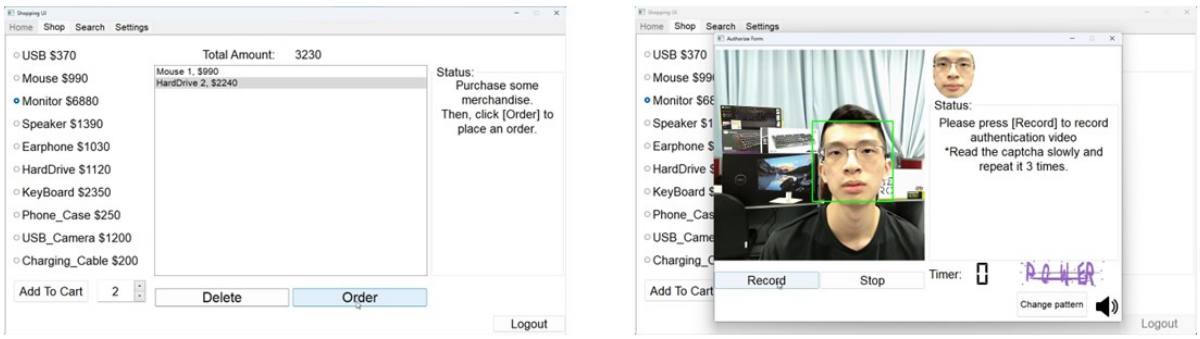


(b) User linked credit card successfully.

Figure 4-4-2: Shopping website account Registration Interface

Shopping Website Registration:

The user enters a username and password to register for the shopping website. Upon successful login, the user can bind their credit card number to their account (Figure 4-4-2).

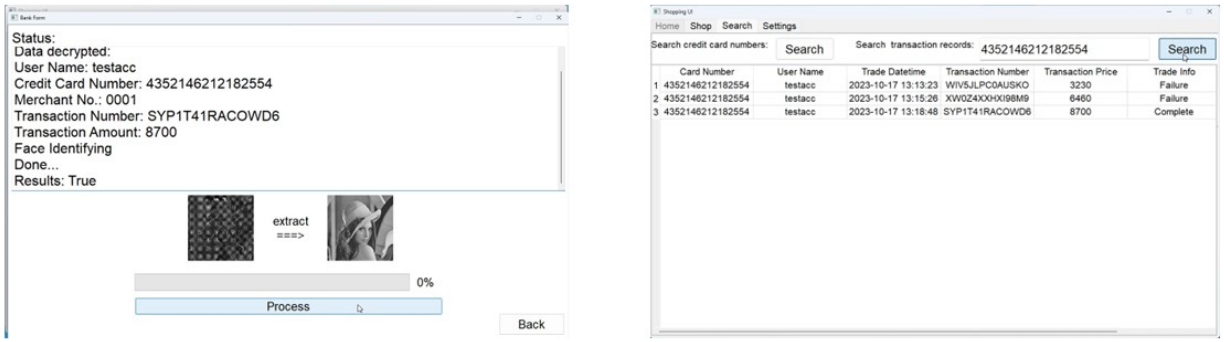


(a) User order merchants (b) User recorded verification video

Figure 4-4-3: User ordering Interface

User ordering:

In Figure 4-4-3(a), the user can add, delete, or modify the quantity of items in the shopping cart before placing an order. In Figure 4-4-3(b), the user records a verification video on the verification website; if the verification passes, the transaction information is sent to the bank.



(a) Bank process transaction informations (b) Look up trsaction records through Shooping website

Figure 4-4-4: Bank and Shopping website Transaction Information Interface

Transaction Processing:

As shown in Figure 4-4-4(a), upon receiving the encrypted image, the bank performs face recognition. If the verification succeeds, an authorization is sent to the e-

commerce website. On the e-commerce website, the user can check recent transaction records by credit card number (Figure 4-4-4(b)).

5. Conclusion and Recommendations

This project designs a third-party secure verification mechanism to replace the conventional CVV-based security method. Using only a face recognition model to verify the cardholder's identity may still be vulnerable to identity spoofing or 2D face image attacks. Therefore, we integrate visual speech recognition as an additional multi-factor authentication method and employ reversible data hiding to ensure data transmission security.

For face recognition, future improvements could involve adopting different neural network architectures to enhance accuracy. For visual speech recognition, improvements are needed in both recognition speed and accuracy. On the verification website, multiple processes must be run simultaneously—FaceNet, MTCNN, VSR recognition, and RDH embedding—which increases computational time. Nevertheless, by combining multiple verification mechanisms, integrating various biometric recognition technologies with data hiding methods, this project ensures that every transaction undergoes rigorous verification, reducing the risk of malicious attacks and credit card fraud, and safeguarding user transaction security.

References

[1] F. Schroff, D. Kalenichenko, and J. Philbin, "FaceNet: A Unified Embedding for Face Recognition and Clustering," p. arXiv:1503.03832doi: 10.48550/arXiv.1503.03832.

[2] M. D. Zeiler and R. Fergus, "Visualizing and Understanding Convolutional Networks," p. arXiv:1311.2901doi: 10.48550/arXiv.1311.2901.

[3] C. Szegedy *et al.*, "Going Deeper with Convolutions," p. arXiv:1409.4842doi: 10.48550/arXiv.1409.4842.

[4] Y. Sun, X. Wang, and X. Tang, "Deep Learning Face Representation by Joint Identification-Verification," p. arXiv:1406.4773doi: 10.48550/arXiv.1406.4773.

[5] P. Ma, S. Petridis, and M. Pantic, "Visual Speech Recognition for Multiple Languages in the Wild," p. arXiv:2202.13084doi: 10.48550/arXiv.2202.13084.

[6] U. Singh and V. B. Singh, "A reversible data hiding scheme for sharing medical data," *Biomedical Signal Processing and Control,* vol. 85, p. 105021, 2023/08/01/ 2023, doi: https://doi.org/10.1016/j.bspc.2023.105021.

[7] D. Yi, Z. Lei, S. Liao, and S. Z. Li, "Learning Face Representation from Scratch," p. arXiv:1411.7923doi: 10.48550/arXiv.1411.7923.

[8] K. Zhang, Z. Zhang, Z. Li, and Y. Qiao, "Joint Face Detection and Alignment Using Multitask Cascaded Convolutional Networks," *IEEE Signal Processing Letters,* vol. 23, pp. 1499-1503, October 01, 2016 2016, doi: 10.1109/lsp.2016.2603342.

[9] J. Deng, J. Guo, Y. Zhou, J. Yu, I. Kotsia, and S. Zafeiriou, "RetinaFace: Single-stage Dense Face Localisation in the Wild," p. arXiv:1905.00641doi: 10.48550/arXiv.1905.00641.

[10] A. Bulat and G. Tzimiropoulos, "How far are we from solving the 2d & 3d face alignment problem?(and a dataset of 230,000 3d facial landmarks)," in *Proceedings of the IEEE international conference on computer vision*, 2017, pp. 1021-1030.

[11] G. B. Huang, M. Mattar, T. Berg, and E. Learned-Miller, "Labeled faces in the wild: A database forstudying face recognition in unconstrained environments," in *Workshop on faces in'Real-Life'Images: detection, alignment, and recognition*, 2008.

[12] T. Afouras, J. Son Chung, and A. Zisserman, "LRS3-TED: a large-scale dataset for visual speech recognition," p. arXiv:1809.00496doi: 10.48550/arXiv.1809.00496.